

실습 #11

Lab11_1

Lab11_1_server.java 코드

```
package week14.lab;

import java.io.*;
import java.net.*;

public class Lab11_1_server {
    public final static int MY_PORT = 5001;
    public static void main(String argv[]) throws Exception
    {
        DatagramSocket ds = null;
        byte buf[] = new byte[256];
        ds = new DatagramSocket(MY_PORT);
        DatagramPacket packet = new DatagramPacket(buf, bu
f.length);
        ds.receive(packet);
        System.out.println("Got from client: ");
        for (int i = 0; i < packet.getLength(); i++) {
            System.out.print((char)buf[i]);
        }
        System.out.println();
        ds.send(packet);
        ds.close();
    }
}
```

Lab11_1_client.java 코드

```
package week14.lab;

import java.io.*;
```

```

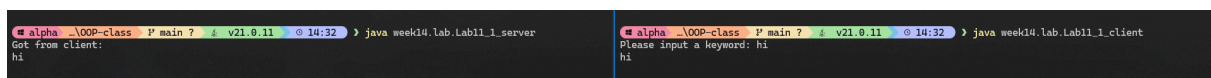
import java.net.*;

public class Lab11_1_client {
    public final static int MY_PORT = 5000;
    public final static int TO_PORT = 5001;
    public static void main(String argv[]) throws Exception
    {
        DatagramSocket ds = new DatagramSocket(MY_PORT);
        byte[] buf = new byte[256];
        System.out.print("Please input a keyword: ");
        int n = System.in.read(buf);
        InetAddress addr = InetAddress.getByName("127.0.0.
1");
        DatagramPacket packet = new DatagramPacket(buf, n,
addr, TO_PORT);
        ds.send(packet);

        DatagramPacket packet2 = new DatagramPacket(buf, bu
f.length);
        ds.receive(packet2);
        buf = packet2.getData();
        for (int i = 0; i < packet2.getLength(); i++) {
            System.out.print((char)buf[i]);
        }
        System.out.println();
        ds.close();
    }
}

```

실행 화면



Lab11_2

Lab11_2_server.java 코드

```

package week14.lab;

import java.io.*;
import java.net.*;

public class Lab11_2_server {
    public static void main(String[] args) {
        try {
            ServerSocket s = new ServerSocket(8189);
            Socket incoming = s.accept();
            BufferedReader in = new BufferedReader(new Input
tStreamReader(incoming.getInputStream()));
            PrintWriter out = new PrintWriter(incoming.getO
utputStream(), true);
            out.println("Hello! Enter BYE to exit.");

            boolean done = false;
            while (!done) {
                String line = in.readLine();
                if (line == null) {
                    done = true;
                } else {
                    System.out.println("Server:" + line);
                    out.println("Echo: " + line);
                    if (line.trim().equals("BYE")) {
                        done = true;
                    }
                }
            }
            incoming.close();
            s.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

Lab11_2_client.java 코드

```
package week14.lab;

import java.io.*;
import java.net.*;

public class Lab11_2_client {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("127.0.0.1", 8189);
            BufferedReader in = new BufferedReader(new Input
tStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(s.getOutputSt
ream(), true);
            BufferedReader stdIn = new BufferedReader(new I
nputStreamReader(System.in));

            String greeting = in.readLine();
            System.out.println(greeting);

            boolean done = false;
            while (!done) {
                String userInput = stdIn.readLine();
                if (userInput == null) {
                    break;
                }
                out.println(userInput);

                String response = in.readLine();
                if (response != null) {
                    System.out.println(response);
                }

                if (userInput.trim().equals("BYE")) {
                    done = true;
                }
            }
        }
    }
}
```

```

        System.out.println("The End");
        s.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
}
}

```

실행 화면

```

alpha \OOP-class P main ? v21.0.11 14:32 in 4s596ms java week14.lab.Lab11_2_ser
ver
Server: hi
Server: hello
Server: BYE
alpha \OOP-class P main ? v21.0.11 14:37 in 21s64ms |

alpha \OOP-class P main ? v21.0.11 14:32 in 15s588ms java week14.lab.Lab11_2_client
Hello! Enter BYE to exit.
hi
Echo: hi
hello
Echo: hello
BYE
Echo: BYE
The End
alpha \OOP-class P main ? v21.0.11 14:37 in 15s588ms |

```

Lab11_3

Lab11_3_server.java 코드

```

package week14.lab;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.io.ObjectInputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Lab11_3_server {
    private static final List<Polygon> serverPlist = Collec
tions.synchronizedList(new ArrayList<>());
    private static ServerPanel canvas;

```

```

    public static void main(String[] args) {
        JFrame frame = new JFrame("Polygon Server (Lab11_3)");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        canvas = new ServerPanel();
        frame.getContentPane().add(canvas, BorderLayout.CENTER);

        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        try (ServerSocket serverSocket = new ServerSocket(9000)) {
            System.out.println("Polygon Server started on port 9000...");
            while (true) {
                Socket socket = serverSocket.accept();
                new ServerHandler(socket).start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static class ServerPanel extends JPanel {
        public ServerPanel() {
            setBackground(Color.black);
            setPreferredSize(new Dimension(600, 400));
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            g.setColor(Color.yellow);
            g.drawString("Total Received Polygons: " + serverPlist.size(), 15, 30);
        }
    }

```

```

        List<Polygon> copy;
        synchronized (serverPlist) {
            copy = new ArrayList<>(serverPlist);
        }

        for (Polygon p : copy) {
            if (p instanceof Circle) {
                Circle c = (Circle) p;
                g.drawOval(c.x - c.radius, c.y - c.radius, c.x + c.radius * 2, c.y + c.radius * 2);
            } else if (p instanceof Rectangle) {
                Rectangle r = (Rectangle) p;
                g.drawRect(r.x, r.y, r.x + r.width, r.y + r.height);
            } else if (p instanceof Line) {
                Line l = (Line) p;
                g.drawLine(l.x, l.y, l.x2, l.y2);
            }
        }
    }
}

private static class ServerHandler extends Thread {
    private final Socket socket;

    public ServerHandler(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try (ObjectInputStream ois = new ObjectInputStream(socket.getInputStream())) {
            ("unchecked")
            ArrayList<Polygon> receivedList = (ArrayList<Polygon>) ois.readObject();
            if (receivedList != null) {
                serverPlist.addAll(receivedList);
                System.out.println("Received " + receivedList.size());
            }
        }
    }
}

```

Lab11_3_client.java 코드

3


```

import javax.swing.JRadioButton;

public class Lab11_3_client {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Polygon Draw (Lab11_3)");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        RubberLinePanel rpanel = new RubberLinePanel();
        frame.getContentPane().add(rpanel);

        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}

class RubberLinePanel_Lab11_3 extends JPanel {
    ArrayList<Polygon> plist = new ArrayList<>();
    Point p1, p2;

    private JRadioButton b1, b2, b3;

    final int CIRCLE = 0;
    final int LINE = 1;
    final int RECTANGLE = 2;
    private int currentType = LINE;

    public RubberLinePanel_Lab11_3() {
        setPreferredSize(new Dimension(600, 300));

        JPanel buttonPanel = new JPanel();
        makeButtonPanel(buttonPanel);
        this.setLayout(new BorderLayout());
        add(buttonPanel, BorderLayout.NORTH);

        add(new DrawPanel(), BorderLayout.CENTER);
    }
}

```

```

        JPanel southPanel = new JPanel();
        JButton sendButton = new JButton("Send");
        sendButton.addActionListener(new SendButtonListener
    ());

        southPanel.add(sendButton);

        JButton clearButton = new JButton("Clear");
        clearButton.addActionListener(new ClearButtonListen
er());
        southPanel.add(clearButton);
        add(southPanel, BorderLayout.SOUTH);
    }

    void makeButtonPanel(JPanel buttonPanel) {
        ButtonGroup buttons = new ButtonGroup();
        b1 = new JRadioButton("Rectangle");
        b2 = new JRadioButton("Oval");
        b3 = new JRadioButton("Line", true);

        DrawButtonListener listener = new DrawButtonListene
r();

        buttons.add(b1);
        buttonPanel.add(b1);
        b1.addActionListener(listener);

        buttons.add(b2);
        buttonPanel.add(b2);
        b2.addActionListener(listener);

        buttons.add(b3);
        buttonPanel.add(b3);
        b3.addActionListener(listener);
    }

    private class SendButtonListener implements ActionListe
ner {

```

```

        public void actionPerformed(ActionEvent event) {
            try (Socket socket = new Socket("127.0.0.1", 9000));

                ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream()) {
                    oos.writeObject(plist);
                    oos.flush();
                    System.out.println("Sent " + plist.size() +
" polygons to server.");
                } catch (Exception e) {
                    System.err.println("Error sending polygons:
" + e.getMessage());
                }

                plist.clear();
                p1 = null;
                p2 = null;
                repaint();
            }
        }

        private class ClearButtonListener implements ActionListener {
            public void actionPerformed(ActionEvent event) {
                System.out.println("clear !!");
                plist.clear();
                p1 = null;
                p2 = null;
                repaint();
            }
        }

        private class DrawButtonListener implements ActionListener {
            public void actionPerformed(ActionEvent event) {
                JRadioButton b = (JRadioButton) event.getSource
();

                String pString = b.getText();

```

```

        System.out.println(pString);
        if (pString.equals("Oval")) {
            currentType = CIRCLE;
        } else if (pString.equals("Line")) {
            currentType = LINE;
        } else {
            currentType = RECTANGLE;
        }
        repaint();
    }
}

private class DrawPanel extends JPanel {
    public DrawPanel() {
        PolygonListener listener = new PolygonListener
();

        addMouseListener(listener);
        addMouseMotionListener(listener);
        setBackground(Color.black);
        setPreferredSize(new Dimension(400, 200));
    }

    private class PolygonListener implements MouseListe
ner, MouseMotionListener {
        public void mousePressed(MouseEvent event) {
            p1 = event.getPoint();
        }

        public void mouseDragged(MouseEvent event) {
            p2 = event.getPoint();
            repaint();
        }

        public void mouseReleased(MouseEvent event) {
            Polygon p;
            if (p1 != null && p2 != null) {
                switch (currentType) {
                    case LINE:

```

```

        p = new Line(p1.x, p1.y, p2.x,
p2.y);

        break;
    case RECTANGLE:
        int x = Math.min(p1.x, p2.x);
        int y = Math.min(p1.y, p2.y);
        int width = Math.abs(p2.x - p1.
x);

        int height = Math.abs(p2.y - p
1.y);

        p = new Rectangle(x, y, width,
height);

        break;
    case CIRCLE:
        int radius = (int) Math.sqrt(Ma
th.pow((p2.x - p1.x), 2) + Math.pow((p2.y - p1.y), 2));
        p = new Circle(p1.x, p1.y, radi
us);

        break;
    default:
        return;
    }
    plist.add(p);
    p1 = null;
    p2 = null;
    repaint();
}

}

public void mouseClicked(MouseEvent event) {}
public void mouseEntered(MouseEvent event) {}
public void mouseExited(MouseEvent event) {}
public void mouseMoved(MouseEvent event) {}
}

public void paintComponent(Graphics page) {
    super.paintComponent(page);
    page.setColor(Color.yellow);
}

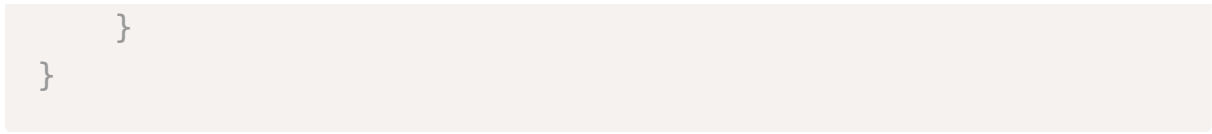
```

```

        for (Polygon p : plist) {
            if (p instanceof Circle) {
                Circle c = (Circle) p;
                page.drawOval(c.x - c.radius, c.y - c.radius, c.radius * 2, c.radius * 2);
            } else if (p instanceof Rectangle) {
                Rectangle r = (Rectangle) p;
                page.drawRect(r.x, r.y, r.width, r.height);
            } else if (p instanceof Line) {
                Line l = (Line) p;
                page.drawLine(l.x, l.y, l.x2, l.y2);
            }
        }

        if (p1 != null && p2 != null) {
            switch (currentType) {
                case LINE:
                    page.drawLine(p1.x, p1.y, p2.x, p2.y);
                    break;
                case CIRCLE:
                    int radius = (int) Math.sqrt(Math.pow((p2.x - p1.x), 2) + Math.pow((p2.y - p1.y), 2));
                    page.drawOval(p1.x - radius, p1.y - radius, radius * 2, radius * 2);
                    break;
                case RECTANGLE:
                    int x = Math.min(p1.x, p2.x);
                    int y = Math.min(p1.y, p2.y);
                    int width = Math.abs(p2.x - p1.x);
                    int height = Math.abs(p2.y - p1.y);
                    page.drawRect(x, y, width, height);
                    break;
            }
        }
    }
}

```



실행 화면

